

Hands On Code Practice with ITensor

Table of Contents (click to jump to each hands-on):

- [Quantum Circuit Gate Evolution / TEBD](#)
- [Functions as tensor networks – function integration](#)
- [The DMRG ground-state-finding algorithm](#)

The TEBD (Gate Evolution) Algorithm – Hands On

Located in codes/gate_evolution_tebd/ folder.

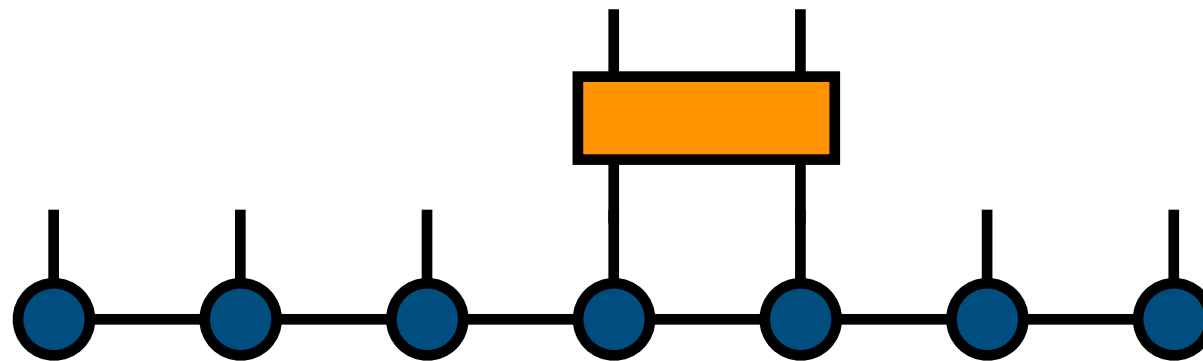
In this exercise, we will finish implementing a code for applying quantum gates to MPS using "TEBD"

Optionally we will finish coding two extensions:

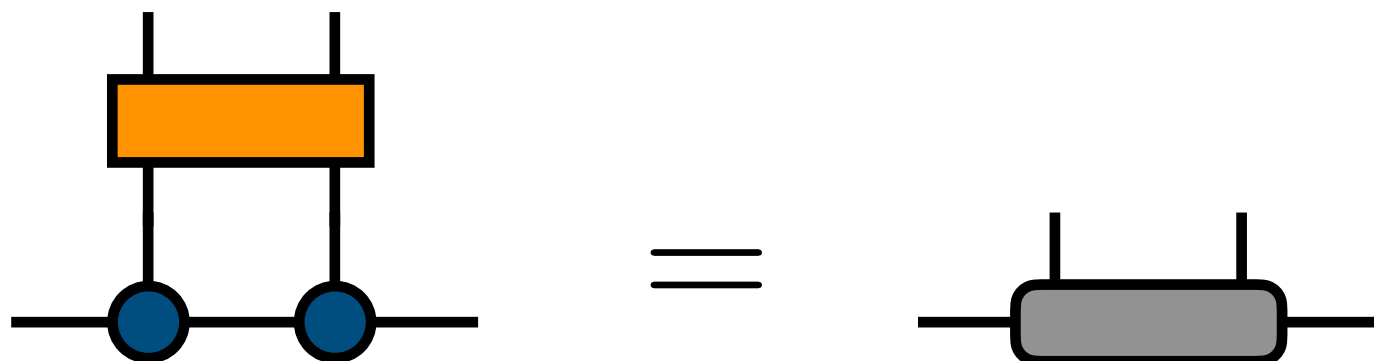
- adapting your code to do Hamiltonian time evolution
- sampling from the final state at the end

Review: TEBD (Gate Evolution) Algorithm

Acting a two-qubit gate on a wavefunction in MPS form

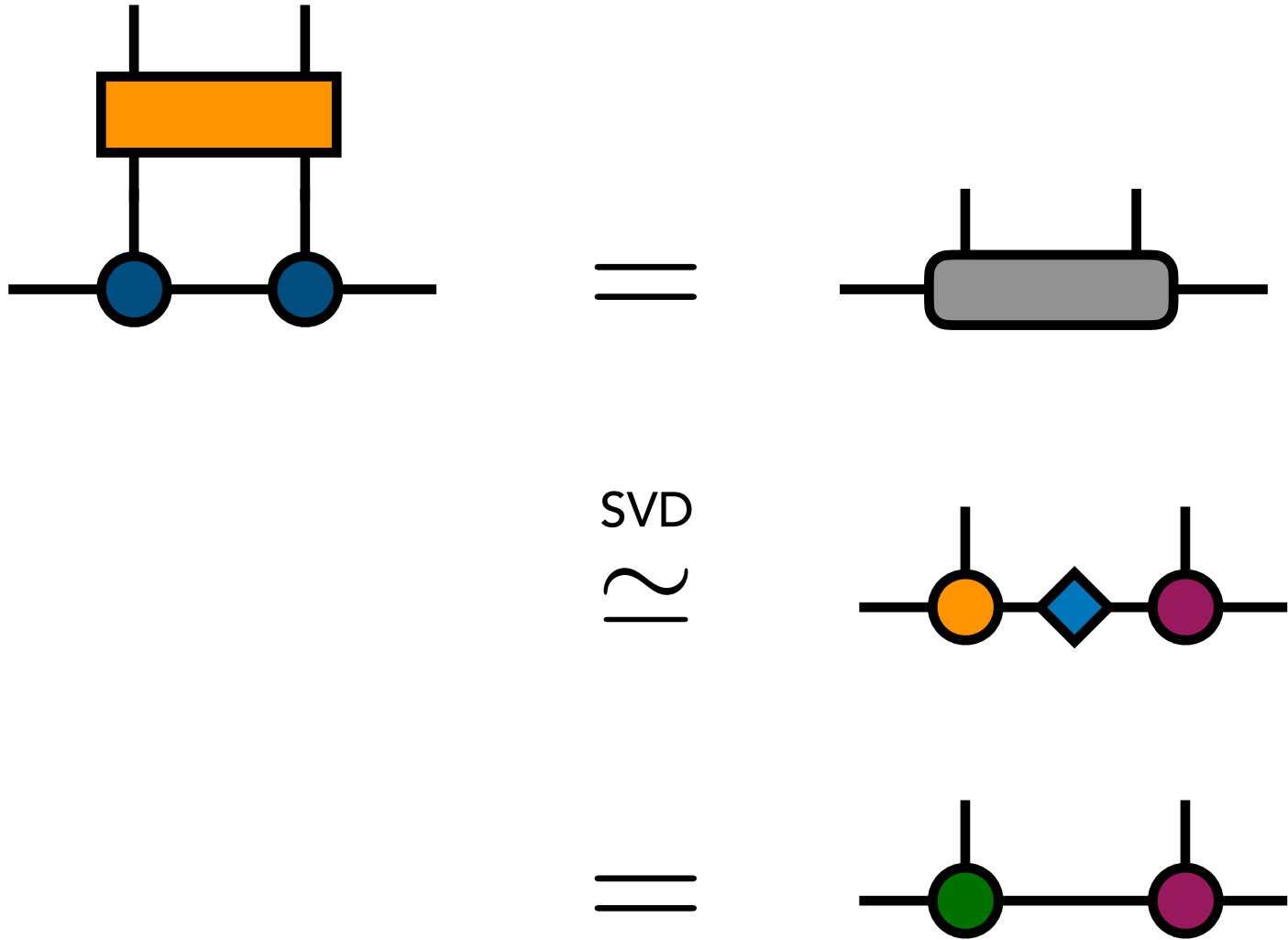


Operates on two tensors:



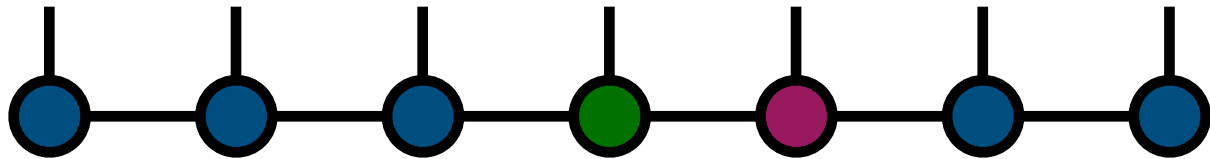
*destroy
MPS form
locally*

Review: TEBD (Gate Evolution) Algorithm



*keep top
 χ values*

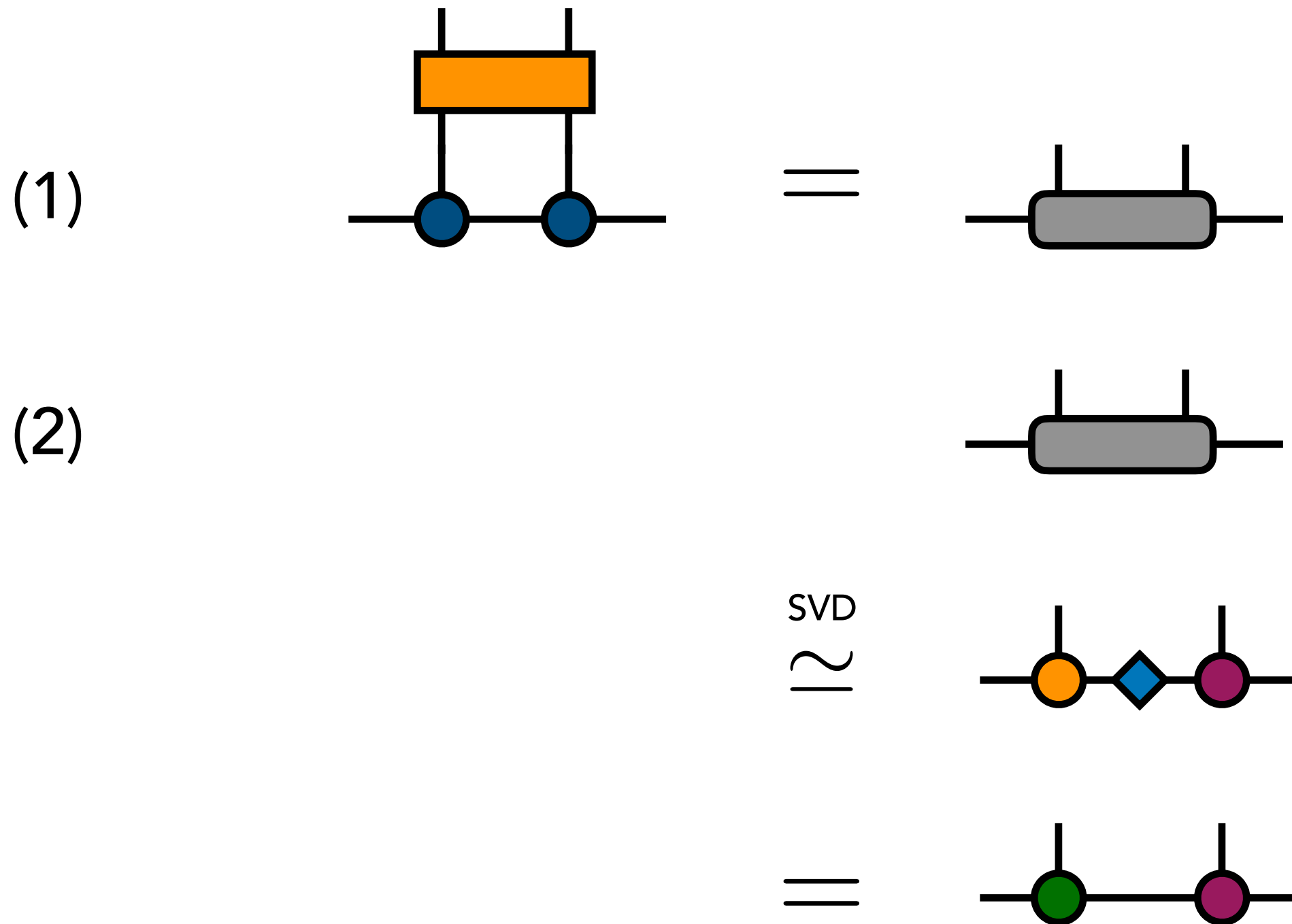
Result:



TEBD (Gate Evolution) Algorithm – Hands On

Around line 20 of `tebd_gate_evolution.jl`

Your task: implement the core steps of TEBD



The TEBD (Gate Evolution) Algorithm – Hands On

To check your work

- load the code:
`julia> include("tebd_gate_evolution.jl")`
- run the `run_tebd()` function
- obtain the bond dimensions (size of new MPS index) adding code using the function `linkdims(psi)` to make sure it is not growing exponentially, though it will still grow. Make sure to pass `cutoff` and `maxdim` truncation parameters. ($1\text{E-}2 > \text{cutoff} > 1\text{E-}12$, `maxdim` anything you wish - sets a maximum bond dimension.)

The TEBD (Gate Evolution) Algorithm – Hands On

To check your work

- load the code:

```
julia> include("tebd_gate_evolution.jl")
```

- load the code:

```
julia> include("01_check_fidelity/check_fidelity.jl")
```

- run the function `check_fidelity(n)` with `n` small, such as `n=10`. It will compare your TEBD code to an exponential, full-state simulator for quality.

The TEBD (Gate Evolution) Algorithm – Hands On

Extension #1: Hamiltonian time evolution

We will make "Trotter gates" that evolve a state using a Hamiltonian – the Heisenberg Hamiltonian $H = \sum_j \vec{S}_j \cdot \vec{S}_{j+1}$

- open the file
02_hamiltonian_evolution/hamiltonian_evolution.jl
- visit [this link](#) to read about making Trotter gates for Hamiltonians – adapt the code into yours (the part about making gates)
- insert code into the time evolution loop (line 28) to measure physical properties of the state - you can plot them or use Makie to make an animation / movie

The TEBD (Gate Evolution) Algorithm – Hands On

Extension #2: quantum state sampling

- review the MPS "perfect sampling" algorithm
<https://tensornetwork.org/mps/algorithms/sampling/>
- your task is to insert code that uses the diagonal elements of the reduced density matrix ρ_j on site j , together with a fixed but random number $r \in (0,1)$, to determine the classical sample value at site j
- can you write your code in a way that will work for any local dimension d , not just $d=2$?
- run the `test_sampling(n;...)` function to verify you get non-trivial samples

The TEBD (Gate Evolution) Algorithm – Hands On

Further challenges:

- replace the randomly generated gates with gates of a specific quantum circuit
- put "swap gates" in between applications of local gates to implement long-range gates

Functions as Tensor Networks – Hands On

Located in codes/function_integration/ folder.

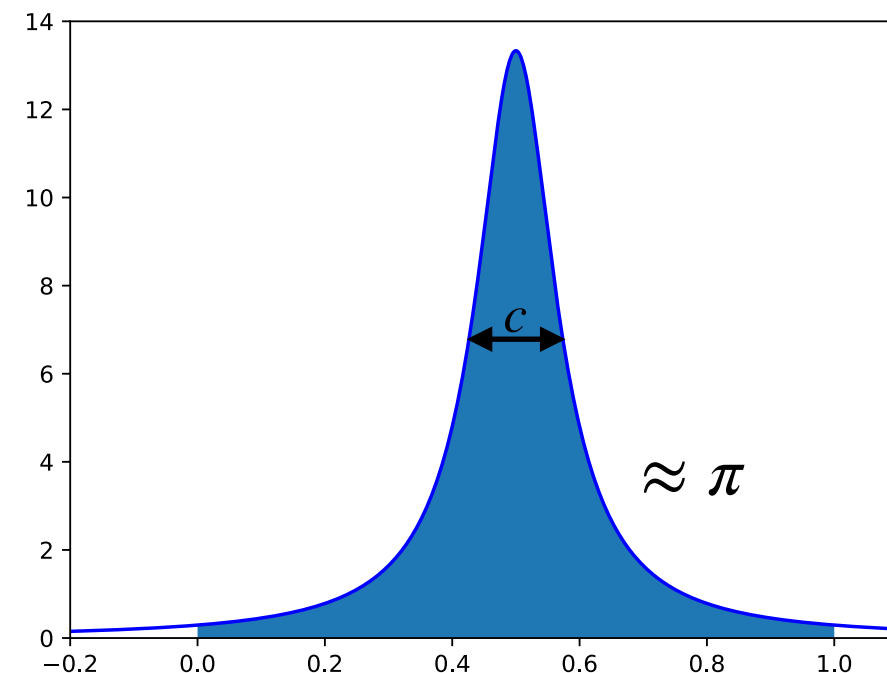
In this exercise, you will use the TCI algorithm
(from a package) to learn a function into MPS form,
then write code to compute the integral $\int_0^1 f(x) dx$

Applications of TCI

Target function: unnormalized Cauchy distribution
area under curve is π

$$f(x) = \frac{c}{(x - \frac{1}{2})^2 + c^2}$$

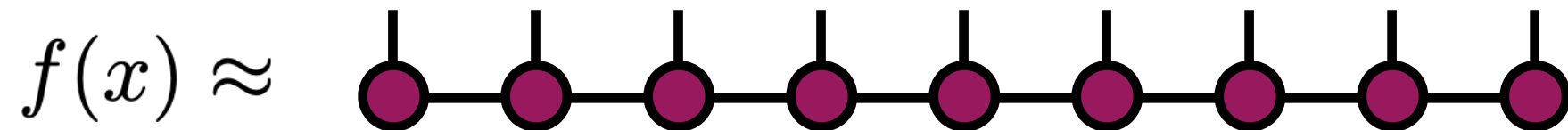
$$\lim_{c \rightarrow 0} \int_0^1 dx f(x) = \pi$$



Applications of TCI

Once we have an MPS or "quantics tensor train" (QTT) version of the function

$$f(x) = \frac{c}{(x - \frac{1}{2})^2 + c^2}$$

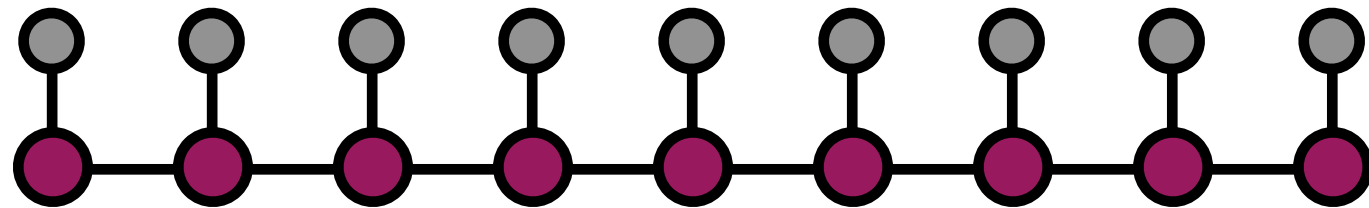


Applications of TCI

We can integrate it from $[0,1)$

by attaching "summation vectors"

$$\int_0^1 f(x) dx \approx$$

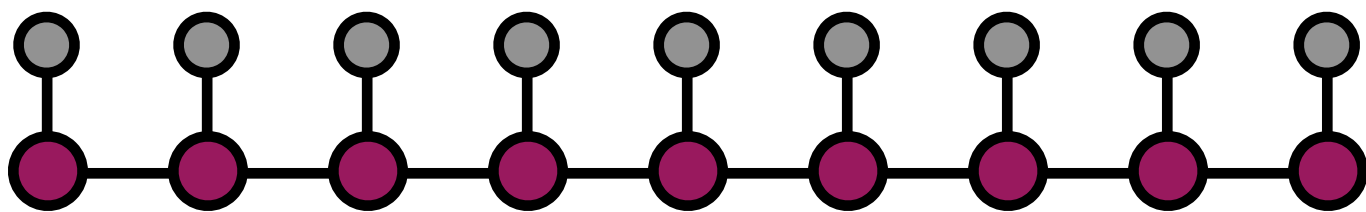


$$\text{⌋} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}$$

$1/2$ comes from the
integral measure dx

Applications of TCI

We can integrate it from $[0,1)$
by attaching "summation vectors"

$$\int_0^1 f(x) dx \approx$$

$$\text{⌋} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix} \quad \begin{array}{l} 1/2 \text{ comes from the} \\ \text{integral measure } dx \end{array}$$

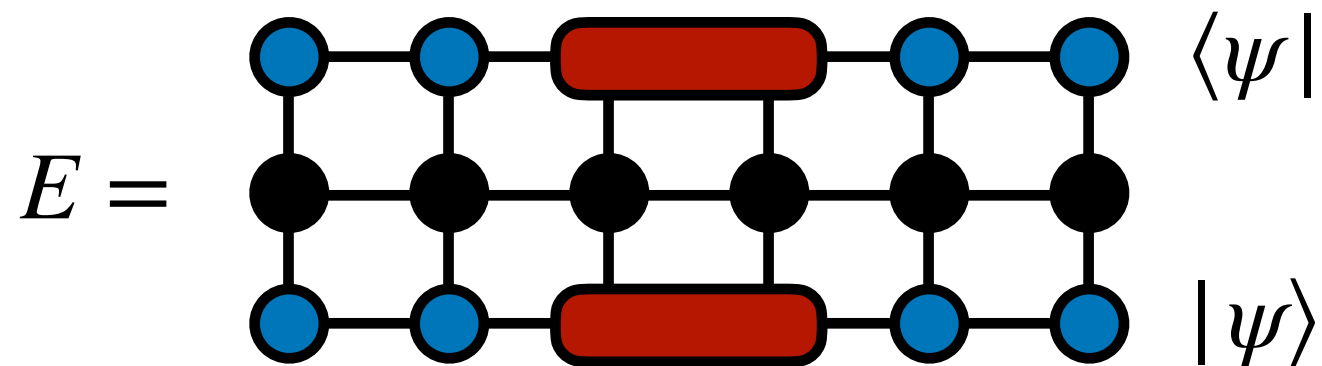
Task: implement the integrate function to compute
the above integral of an MPS function approximation

The DMRG Algorithm – Hands On

Located in [codes/dmrg/](#) folder.

In this exercise, we will finish implementing a code for the **DMRG** algorithm

Specifically the *two-site* variant of DMRG



The DMRG Algorithm

DMRG algorithm

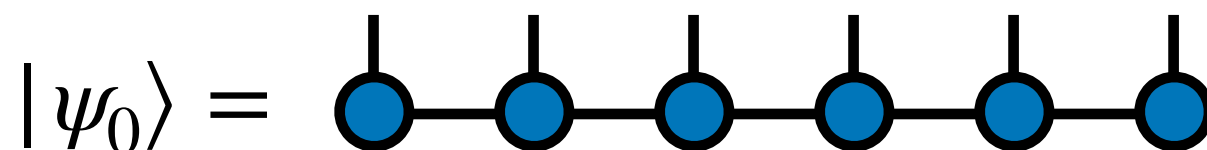
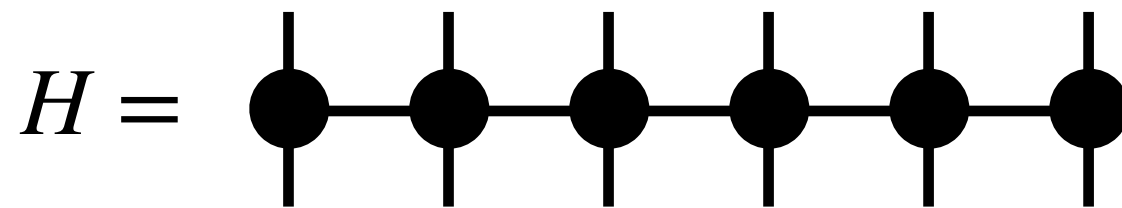
Assume we can write H as a tensor network

$$H = \text{---} \bullet \text{---} \bullet \text{---} \bullet \text{---} \bullet \text{---} \bullet \text{---} \bullet \text{---}$$

The DMRG Algorithm

DMRG algorithm

DMRG finds its ground state (minimum-energy eigenvector) as an MPS tensor network



The DMRG Algorithm

DMRG algorithm

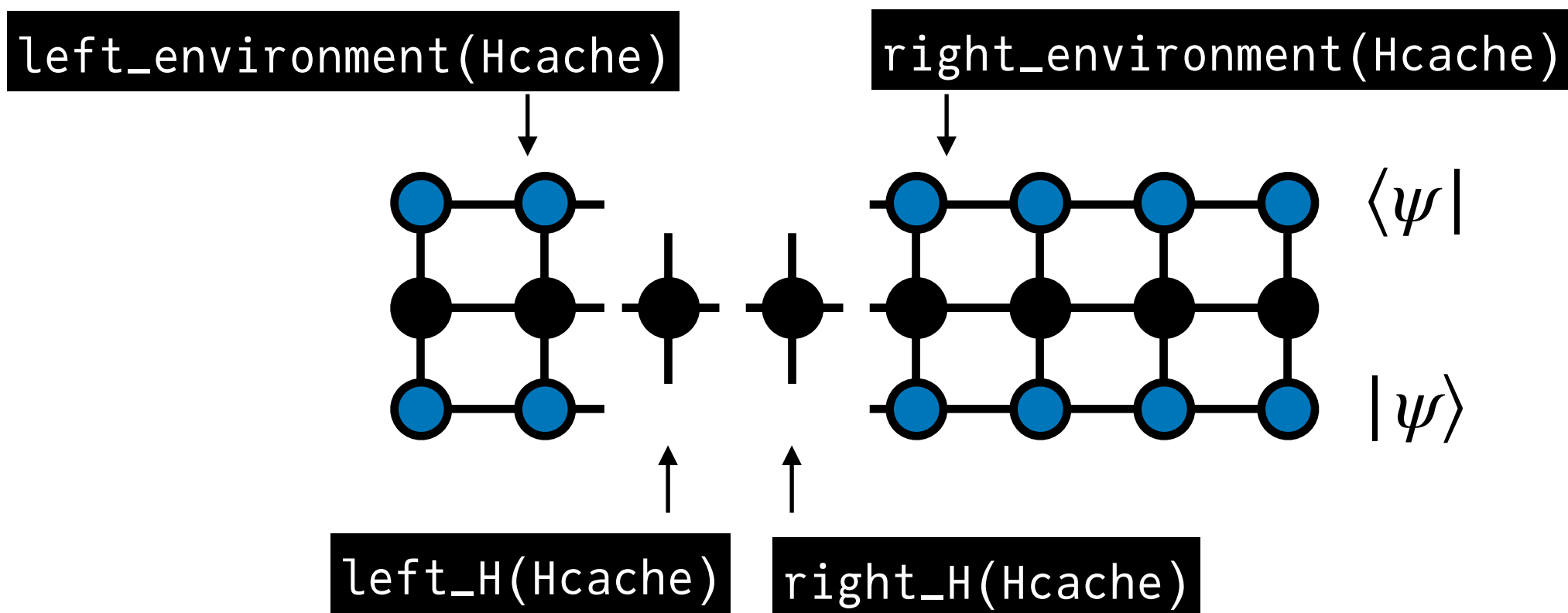
Energy is

$$E = \begin{array}{cccccc} \bullet & \bullet & \bullet & \bullet & \bullet & \bullet & \langle \psi | \\ | & | & | & | & | & | & \\ \bullet & \bullet & \bullet & \bullet & \bullet & \bullet & \\ | & | & | & | & | & | & \\ \bullet & \bullet & \bullet & \bullet & \bullet & \bullet & | \psi \rangle \end{array}$$

The DMRG Algorithm – Hands On

Code includes a pre-written type called MPOCache

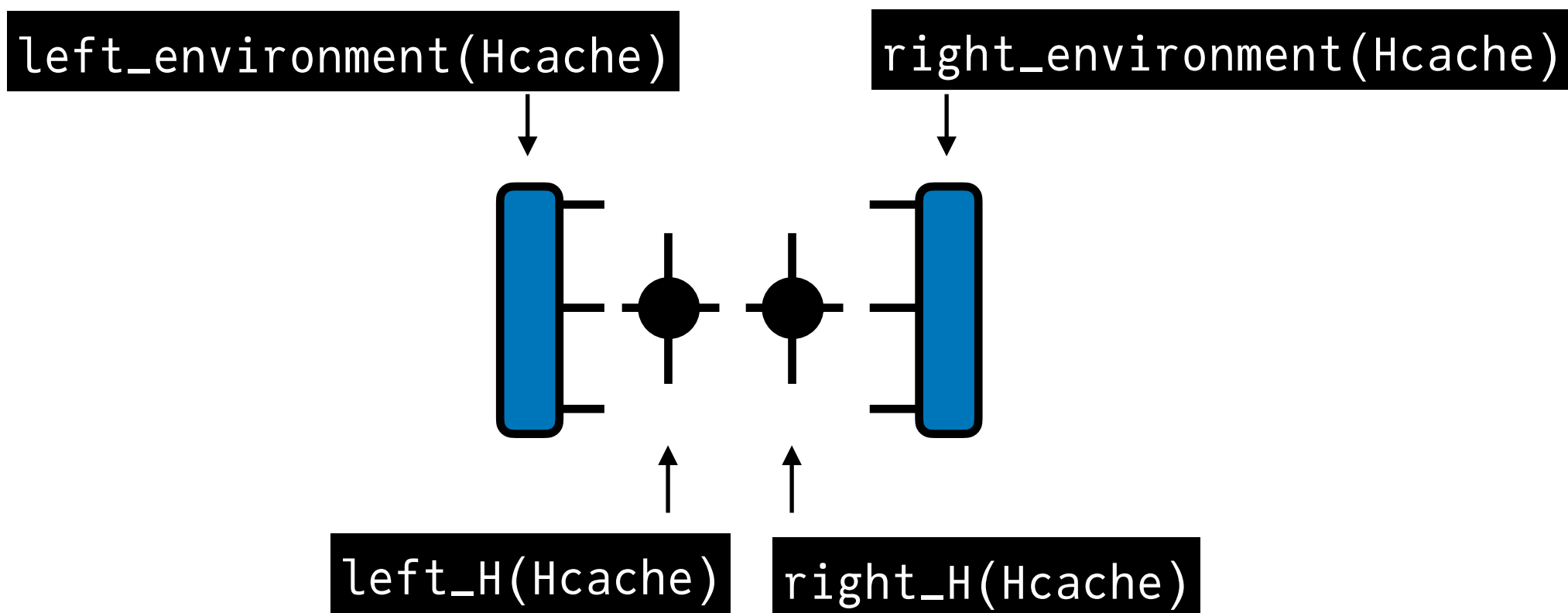
Calling `Hcache = position(Hcache,H,psi,bond)`
will build left and right environments around a
two-site $\text{bond}=(j,j+1)$:



The DMRG Algorithm – Hands On

Code includes a pre-written type called MPOCache

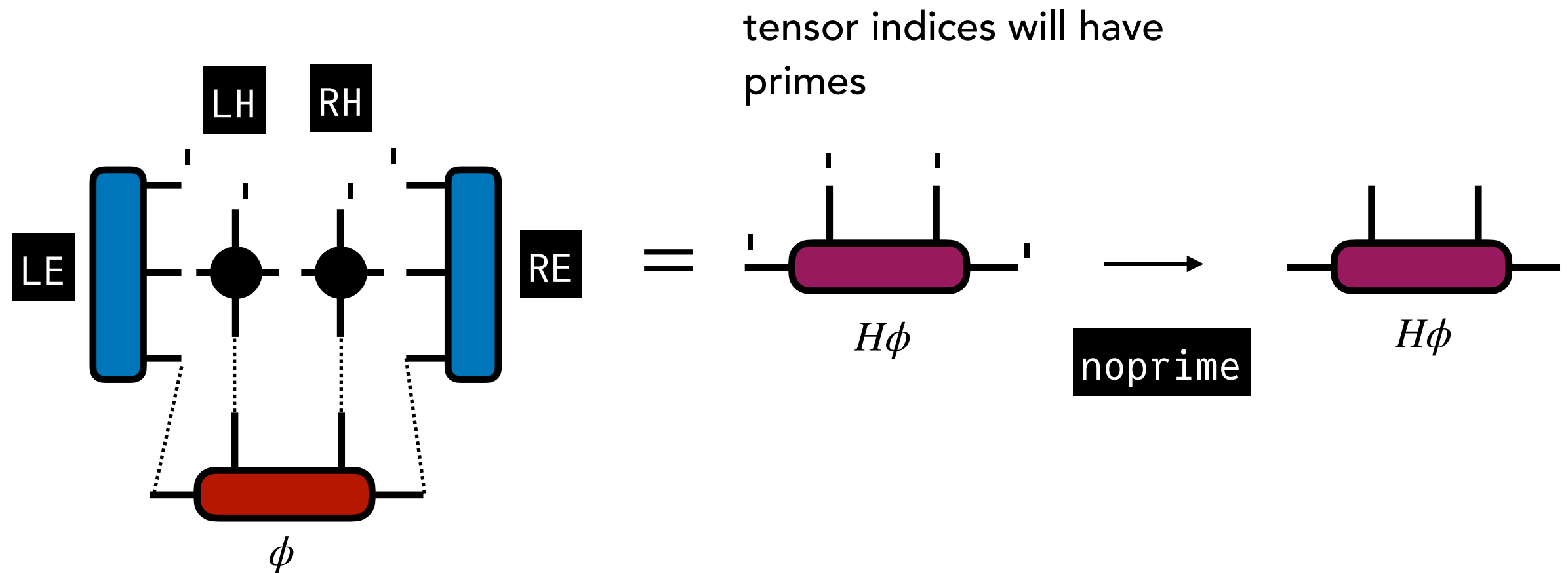
Calling `Hcache = position(Hcache,H,psi,bond)`
will build left and right environments around a
two-site bond= $(j,j+1)$:



The DMRG Algorithm – Hands On

Task 1: finish implementing the `mult(Hcache,phi)` function

What it should do:

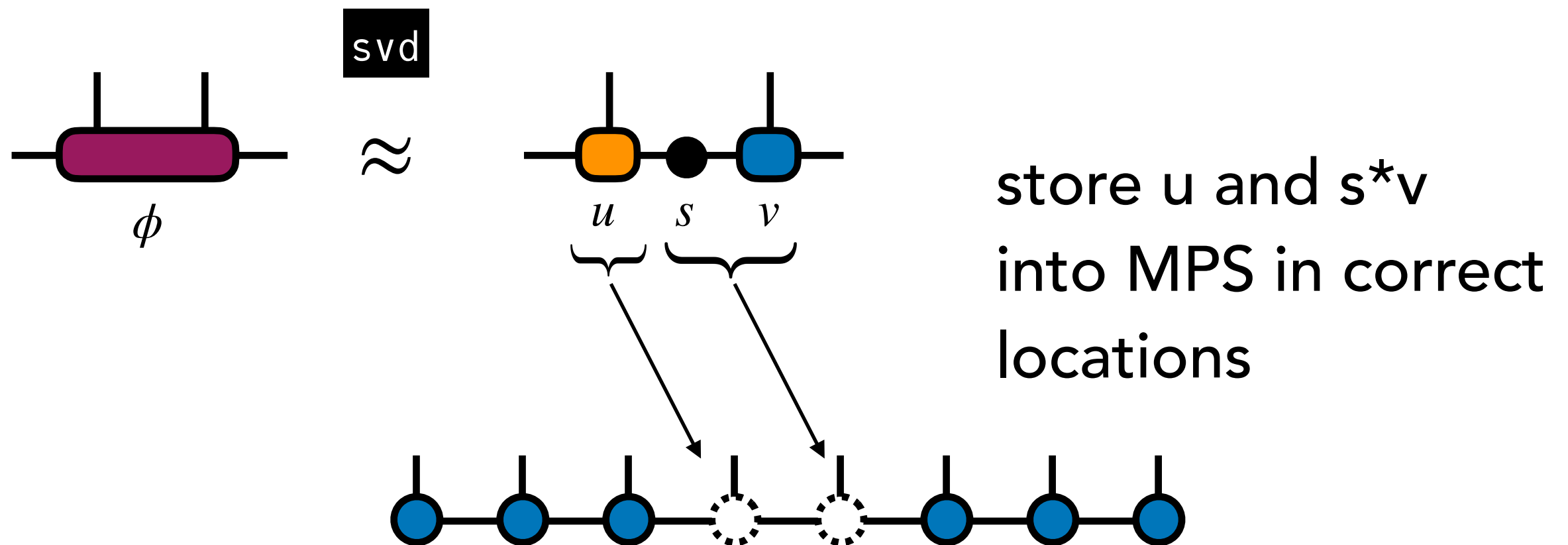


and return $H\phi$

The DMRG Algorithm – Hands On

Task 2: SVD apart the two-site wavefunction to put back into the MPS

What you should do:



The DMRG Algorithm – Hands On

Task 3: further improvements to your DMRG code

- check that energy is correct
- use the expect function compute `expect(psi, "Sz")` and plot. What is going on at the boundaries?
- improve printing options
 - print the bond the code is on
 - print the MPS bond dimension after the SVD
 - allow changing the amount of printing through a keyword option
- collect data during DMRG run (e.g. local properties of MPS) and use Makie.jl package to make a movie!

